Case Study

RTL-to-GDSII Implementation of a Streaming Harris Corner Detection Accelerator

ASIC Design (BEVD301L) • Fall Semester 2026–27 • Extension of the Group Final Project

Total Marks: 10

Team Details

Name	Registration Number	Lab Slot
Saket Sodani	24BVD0069	L9 + L10
Advay Gopalakrishnan	24BVD0191	L21 + L22
Swajit Jain	24BVD0022	L21 + L22
Krithik Sai	24BVD0039	L21 + L22
Arrnav Trisal	24BVD0064	L21 + L22
Ashmit Singh	24BVD0027	L21 + L22

Literature Survey

This section shows the technical literature our group studied to understand existing approaches to hardware Harris corner detection. It covers papers from 1988 to 2023 under three themes: the algorithm and its hardware-friendly formulation; streaming FPGA architectures and their memory organisation; and techniques that reduce bit width, area and power. Within each subsection recent work is discussed first, followed by earlier and foundational work.

Papers Reviewed

Table 1 lists the eleven papers and one vendor reference design reviewed by the group.

Table 1: Papers Reviewed

Ref.	Title	Authors	Publication venue	Year	DOI / Link
[1]	Real Time FPGA Implementation of an Efficient High Speed Harris Corner Detection Algorithm Based on High-Level Synthesis	R. Ghodhbani, T. Saidani, A. Alhomoud, A. Alshammari, R. Ahmed	Engineering, Technology & Applied Science Research, vol. 13, no. 6, pp. 12169–12174	2023	10.48084/etasr.6306
[2]	Real Time FPGA Implementation of a High Speed and Area Optimized Harris Corner Detection Algorithm	P. Sikka, A. R. Asati, C. Shekhar	Microprocessors and Microsystems, vol. 80, art. 103514	2021	10.1016/j.micpro.2020.103514
[3]	Threshold-Guided Design and Optimization for Harris Corner Detector Architecture	B. A. Jasani, S.-K. Lam, P. K. Meher, M. Wu	IEEE Trans. Circuits and Systems for Video Technology, vol. 28, no. 12, pp. 3516–3526	2018	10.1109/TCSVT.2017.2757998
[4]	Lowering Dynamic Power in Stream-Based Harris Corner Detection Architecture	S.-K. Lam, R. K. Bijarniya, M. Wu	Int. Conf. Field-Programmable Technology (ICFPT)	2017	ntu.edu.sg/home/ASSKLam (author copy)
[5]	An Efficient FPGA Implementation of the	T. L. Chao, K. H. Wong	IAPR Int. Conf. Machine Vision Applications	2015	mva-org.jp; code: github.com/chaotaklon

Ref.	Title	Authors	Publication venue	Year	DOI / Link
	Harris Corner Feature Detector		(MVA), pp. 89–93		
[6]	FPGA-Based Flexible Hardware Architecture for Image Interest Point Detection	A. Hernandez-Lopez, C. Torres-Huitzil, J. J. Garcia-Hernandez	Int. J. Advanced Robotic Systems, vol. 12	2015	10.5772/61058
[7]	A Multi-Resolution FPGA-Based Architecture for Real-Time Edge and Corner Detection	P. R. Possa, S. A. Mahmoudi, N. Harb, C. Valderrama, P. Manneback	IEEE Trans. Computers, vol. 63, no. 10, pp. 2376–2388	2014	10.1109/TC.2013.130
[8]	An FPGA Sliding Window-Based Architecture Harris Corner Detector	A. Amaricai, C.-E. Gavriliu, O. Boncalo	24th Int. Conf. Field Programmable Logic and Applications (FPL), pp. 1–4	2014	10.1109/FPL.2014.6927402
[9]	Pipelining Harris Corner Detection with a Tiny FPGA for a Mobile Robot	M. F. Aydogdu, M. F. Demirci, C. Kasnakoglu	IEEE Int. Conf. Robotics and Biomimetics (ROBIO), pp. 2177–2184	2013	10.1109/ROBIO.2013.6739792
[10]	Multilayered Image Processing for Multiscale Harris Corner Detection in Digital Realization	P.-Y. Hsiao, C.-L. Lu, L.-C. Fu	IEEE Trans. Industrial Electronics, vol. 57, no. 5, pp. 1799–1805	2010	10.1109/TIE.2010.2040556
[11]	A Combined Corner and Edge Detector	C. Harris, M. Stephens	Proc. 4th Alvey Vision Conference, pp. 147–151	1988	10.5244/C.2.23
[12]	Corner Detector (FAST / Harris) block, Vision HDL Toolbox	MathWorks	Product documentation (reference design)	–	mathworks.com/help/visionhdl

Summary of Each Paper

In 2023, Ghodhbani *et al.* [1] addressed real-time corner detection on high-resolution video, which overloads processor-only systems. They modelled the Harris filter in Simulink, generated the hardware with HDL Coder and the Vision HDL Toolbox, and integrated it as an IP core on a Zynq-7020 with an HDMI camera, with the ARM core handling control. The system processes 1280×1024 video at 60 fps with a 148.5 MHz system clock, using about 11% of the LUTs, 10% of the flip-flops, 27% of the block RAM and 21 DSP slices. Their verification compares the RTL output with a bit-accurate fixed-point MATLAB model, which is the same method we use with our Python reference model.

In 2021, Sikka *et al.* [2] targeted a Harris implementation that is both fast and small. They used a high-level synthesis approach in which every intermediate data node is given its own application-specific bit width; RTL was generated with MATLAB HDL Coder and implemented on a ZedBoard with Vivado. Verified on a live video stream, the design reports a lower gate count and higher speed than earlier implementations on the same board. The lesson for us is that area savings come from assigning a width to each node rather than one wide word throughout. We apply the same per-node width analysis, but by hand in SystemVerilog, so that every register in the netlist can be traced back to the RTL.

In 2018, Jasani *et al.* [3] studied the bit-width growth of the Harris pipeline, whose response is about six times wider than the input pixel. Because only the comparison R>T matters, they propagated truncation errors from the gradient stage to R analytically and removed low-order bits while the worst-case error stayed below the threshold T. Stage widths fell by about 45% on average (the response from 45 to 25 bits). In TSMC 65 nm the area fell by about 35% and the maximum clock rose from 224 MHz to 327 MHz or 500 MHz depending on the variant, at an average loss of only 0.57% in repeatability. This is the most relevant study for an ASIC project: it confirms that a full-precision datapath is the correct verifiable baseline and that the response multipliers set the critical path.

In 2017, Lam *et al.* [4] addressed dynamic power in streaming Harris pipelines, where every stage switches on every pixel and the row buffers, which dominate power, cannot be clock-gated. An inexpensive early estimate of the gradient products identifies pixels that are clearly not corners, using a threshold taken from the previous frame; their gradients are forced to zero so that the downstream

multipliers and adders stop toggling. On a Cyclone V the method saved about 10% dynamic power on average (7.6–13.2%) for under 0.6% extra area, with 68–94% of pixels rejected early and almost no loss of accuracy. It shows where the power of a design like ours goes and gives a ready optimisation for later versions.

In 2015, Chao and Wong [5], whose open-source code [13] was cited in our proposal, built a low-cost Harris detector on a ZedBoard for a 640×480, 60 fps camera. Grayscale conversion uses shifts, line buffers in flip-flops and block RAM form the window, only three entries of the symmetric matrix M are computed, and k≈0.04 is replaced by 5/128, a multiply and shift. Non-maximum suppression is replaced by a binary feature window, and the threshold is doubled or halved each frame to control the corner count. The detector uses 977 LUTs, 662 registers, five 18 kb BRAMs and 110 DSP blocks, and runs at 60 fps on hardware. Its results match OpenCV except for false corners on the last scan line, caused by mixing the end of one frame with the start of the next. We adopt three ideas: exploiting the symmetry of M, a small fixed-point k, and explicit border handling to avoid exactly that artefact.

In the same year, Hernandez-Lopez *et al.* [6] noted that hardware interest-point detectors usually support only one algorithm. They built parallel, configurable processing elements for generic window operations, with buffering that pipelines operators at coarse grain, and ran both Harris and SUSAN on the same fabric. A Virtex-6 at 50 MHz achieves 49.45 Mpixel/s, or 161 fps at VGA. This modular separation of window generation from computation is reflected in our separate `window3x3` and `boxsum5x5` modules.

In 2014, Possa *et al.* [7] proposed a flexible FPGA architecture shared by the Canny edge and Harris corner detectors. Its neighbourhood extractors and threshold operators are parameterised at run time, it accepts variable input resolution with reduced latency and memory, and algorithmic simplifications cut complexity without loss of reliability. Compared with an optimised GPU version and a CPU reference, the FPGA reaches competitive throughput at a much lower clock frequency, with a clear power advantage and reliable results on noisy images. It supports our choice to make image width, height, k and the threshold parameters rather than fixed constants.

Also in 2014, Amaricai *et al.* [8] observed that Harris is a chain of window operations whose intermediate results are costly to keep in external memory or large BRAM line buffers. Their sliding processing window keeps each stage's data local to the next stage. On Spartan-6 the design exceeds 130 fps at 1280×720 with significantly fewer BRAMs than other approaches. This made clear that line storage, not arithmetic, dominates the cost of a Harris engine. For our ASIC this matters even more: without SRAM macros in the course flow, every buffered bit becomes a flip-flop.

In 2013, Aydogdu *et al.* [9] built a three-module Verilog pipeline on a small Virtex-5 (XC5VLX50) for a mobile robot's 640×480 stereo camera. Raw and processed images shared a single DDR2 memory that allows one access at a time. The design fits in under 75% of the device at 100 MHz but detects corners at only 0.33 pixel per clock (54 Hz for the stereo pair). The bottleneck was external memory access, not computation, which is why our design keeps all intermediate data on chip.

In 2010, Hsiao *et al.* [10] analysed the data flow of a multiscale Harris front end followed by shape-context matching, where each layer waited for the previous one and repeatedly re-read memory. Their FPGA “visual pipeline” combines parallel and pipelined processing so that each layer consumes its predecessor's output as it is produced, giving a real-time hardware/software recognition system. It is an early argument for the stage-to-stage streaming that organises our design.

The foundational work is that of Harris and Stephens [11] (1988). They improved Moravec's detector, which was noisy and tested only four shift directions, by expanding the local auto-correlation analytically into a 2×2 matrix M of window-weighted gradient products. The eigenvalues of M classify a point as flat, edge or corner, and the response $R=\det(M)-k\,\mathrm{trace}(M)^2$ avoids computing them. R is large and positive at corners and negative along edges. The use of a determinant and trace, without square roots or division, is what makes the detector practical in integer hardware. Finally, the MathWorks Vision HDL Corner Detector [12] serves as an industrial reference: it uses 1×3 and 3×1 gradient kernels, 5×5 circular Gaussian windows and a three-multiplier response stage, and keeps a 44-bit full-precision metric for 8-bit input. On a ZC706 it uses 510 LUTs and 894 registers at 364 MHz. It independently confirms both our bit-growth estimate and our choice of a 5×5 integration window.

Comparison of Existing Approaches

Table 2 compares the hardware papers. They fall into hand-written streaming RTL designs [3], [4], [5], [6], [8], [9] and model-based HLS flows [1], [2], [12]; only [3] reports standard-cell ASIC results. The algorithm paper [11] and the system paper [10] report no comparable hardware metrics and are omitted. Figures are as reported on different devices and resolutions, so they show design trends rather than a like-for-like ranking.

Table 2: Comparison of Existing Hardware Harris Implementations ("n.r." = not reported)

Ref.	Design method	Platform	Window / smoothing	Arithmetic and k	Corner selection	Clock	Reported throughput	Resources / area / power
[1]	HLS (Simulink, Vision HDL)	Zynq-7020	n.r.	"Optimum" bit width	n.r.	148.5 MHz	1280×1024 at 60 fps	5,917 LUT, 10,981 FF, 37 BRAM, 21 DSP
[2]	HLS (HDL Coder)	Zynq ZedBoard	n.r.	Per-node bit widths	n.r.	n.r.	Real-time video	Lower gate count than prior ZedBoard designs
[3]	RTL, analytic truncation	TSMC 65 nm ASIC; Cyclone IV	5 row buffers, Gaussian	Threshold-guided truncation, R: 45→25 bits	Threshold + NMS	327–500 MHz (ASIC)	1,067–1,627 fps at 640×480 (ASIC)	≈0.71 mm^2, 164–249 mW; 35% less area
[4]	Verilog RTL	Cyclone V	Row buffers, smoothed products	Early zeroing of non-corners	Threshold from previous frame	70 MHz	Streaming, 640×480	33,318 ALMs; ≈10% less dynamic power
[5]	Verilog RTL	Zynq xc7z020	Differences; window sum	k=5/128 (mult. + shift); 3 terms of M	Binary feature window, adaptive threshold	n.r.	640×480 at 60 fps (144 fps sim.)	977 LUT, 662 FF, 5 BRAM, 110 DSP
[6]	RTL, configurable window PEs	Virtex-6	Generic window operators	n.r.	n.r.	50 MHz	161 fps at VGA (49.45 Mpixel/s)	n.r.
[7]	RTL, run-time parameterised	FPGA (vs. GPU, CPU)	Parameterisable neighbourhoods	Simplified algorithm	Parameterisable threshold	Lower than GPU	Competitive with GPU	Clear power advantage over GPU
[8]	RTL, sliding window	Spartan-6, Virtex-5	Sliding processing window	n.r.	n.r.	n.r.	>130 fps at 1280×720	Far fewer BRAMs than prior work
[9]	Verilog RTL, 3-stage pipeline	Virtex-5 + DDR2	n.r.	n.r.	n.r.	100 MHz	0.33 pixel/clock; 54 Hz stereo 640×480	<75% of device
[12]	HDL Coder library block	ZC706	1×3/3×1 gradients; 5×5 Gaussian	Full precision, 44-bit metric	Threshold, metric output	364 MHz	n.r.	510 LUT, 894 FF, 0 DSP, 0 BRAM
Ours	SystemVerilog RTL, Cadence flow	Course standard-cell library	3×3 Sobel; 5×5 box sum	Exact integer, 54-bit R; k in Q0.8	Threshold, binary flag (no NMS in v1)	100 MHz target	1 pixel/clock	From Genus / Innovus reports

Three contrasts stand out. *Memory strategy:* [9] relied on external DDR and reached only 0.33 pixel per clock, whereas [3], [4], [5], [8] keep all data on chip in row buffers and sustain one pixel per clock; on-chip line storage is the price of full-rate streaming. *Precision:* [5] and [3] reduce arithmetic cost (a shift-friendly k; truncation bounded by the threshold), while [12] keeps full precision; only [3] quantifies the trade-off in silicon, about 35% area for 0.57% repeatability. *Design method:* HLS flows [1], [2], [12] are quick to build but hide the register structure, while hand-written RTL [3], [4], [5] gives the control over pipeline registers and widths that synthesis, equivalence checking and timing closure in an ASIC flow require.

Key Learnings

The survey shaped our understanding of the project in six ways:

1. **Streaming needs on-chip line buffers.** Throughput collapses when intermediate data go through external memory [9]; row buffers sized to one image width give one pixel per clock [5], [8]. Our buffers are therefore one row deep and the input is never framed.

2. **Line storage, not arithmetic, dominates cost** [4], [8]. In an ASIC without SRAM macros every buffered bit is a flip-flop, so buffer width must be justified and image width must remain a parameter.
3. **Bit growth is large and must be bounded explicitly.** The response is about six times wider than the pixel [3], [12]. We derive every width from worst-case bounds so no stage can overflow.
4. **The response multipliers set the critical path** [3]. If our 10 ns target is not met, the response stage is the first candidate for extra pipelining or threshold-guided truncation.
5. **Cheap approximations of k and window weights work.** k=5/128 [5] and simple windows give results close to software; our Q0.8 value 10/256 equals 5/128 exactly.
6. **Borders and verification need explicit design.** Frame-boundary artefacts [5] and fixed-point mismatches are avoided by masking incomplete windows and comparing RTL against a bit-accurate model [1].

Project Description

Based on the literature survey, this section explains the project our group is undertaking.

Problem Statement

Embedded vision systems need corner features at video rate, but computing the Harris response in software needs two gradient convolutions, three products, three window sums and a response evaluation for every pixel, plus repeated memory traffic. This is too much for the low-power processors in robots and smart cameras. Almost all of the hardware solutions we surveyed are FPGA prototypes, many generated through HLS, and only one of them [3] reports standard-cell ASIC results. Our problem is to design a Harris corner detector as an ASIC block that sustains one pixel per clock with deterministic latency, avoids arithmetic overflow by construction, and is carried through a complete, reproducible RTL-to-GDSII flow with formal and timing sign-off evidence.

Proposed Design

The proposed system, `harris_top`, is a streaming Harris corner detection accelerator written in synthesizable SystemVerilog. It accepts a raster-ordered stream of 8-bit grayscale pixels with a valid strobe and, for each pixel whose full neighbourhood is available, outputs a binary corner decision together with the pixel's original (x,y) coordinates. Its major features are:

- one pixel accepted per clock in steady state, with a low `pixel_valid` acting as a stall of the entire pipeline;
- frame-free operation: only one-row line buffers are stored, never a full image;
- 3×3 Sobel gradients, exact gradient products, 5×5 box integration and the integer Harris response;
- compile-time parameters for image width and height, the Harris constant k (Q0.8) and the response threshold;
- border suppression, so only pixels with complete 3×3 and 5×5 neighbourhoods produce valid outputs;
- coordinate and valid signals pipelined with the data, so each output is self-describing;
- exact integer arithmetic through the threshold, so the RTL reproduces the Python reference model bit for bit.

The RTL, reference model, testbenches and Cadence scripts are maintained in the project repository [14].

Proposed Architecture

Figure 1 shows the block diagram. The datapath is a linear pipeline of six modules; image coordinates and a valid bit travel alongside the data through every stage.

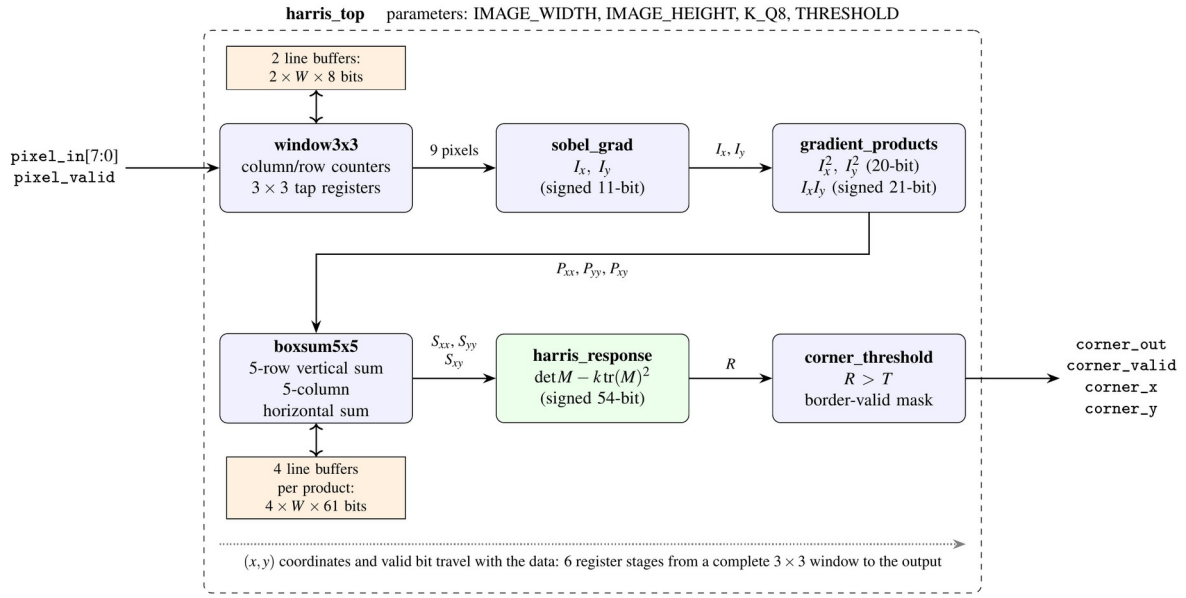


Figure 1: Block diagram of the proposed one-pixel-per-clock streaming Harris corner detection accelerator (W = image width).

The role of each block is as follows.

window3x3 counts columns and rows of the incoming raster, stores the two previous rows in line buffers of width W, and shifts three horizontal taps per row to present a 3×3 neighbourhood every accepted cycle. It also generates the centre-pixel coordinates and marks windows that would extend past the image edge as invalid.

sobel_grad computes the horizontal and vertical gradients with the Sobel kernels using only shifts, additions and subtractions:

$$I_x = (p_{02} + 2p_{12} + p_{22}) - (p_{00} + 2p_{10} + p_{20}), \quad (1)$$

$$I_y = (p_{20} + 2p_{21} + p_{22}) - (p_{00} + 2p_{01} + p_{02}). \quad (2)$$

For 8-bit pixels $|I_x|,|I_y| \le 1020$, which fits a signed 11-bit word.

gradient_products forms the three distinct entries of the structure tensor, $P_{xx}=I_x^2$, $P_{yy}=I_y^2$ and $P_{xy}=I_xI_y$; the fourth entry is not computed because M is symmetric [5].

boxsum5x5 integrates each product over a 5×5 window. Four line buffers per product hold the previous four rows, a five-term vertical sum is formed each cycle, and a five-entry shift register adds the last five vertical sums:

$$S_{ab}(x, y) = \sum_{i=-2}^{2} \sum_{j=-2}^{2} P_{ab}(x + i, y + j), \quad ab \in \{xx, yy, xy\}. \quad (3)$$

A uniform window replaces the Gaussian of [11] so that no multipliers are needed; because the scale factor is common to all terms, it is absorbed into the threshold.

harris_response evaluates the response on the matrix $M = [S_{xx}, S_{xy}; S_{xy}, S_{yy}]$ as

$$R = S_{xx}S_{yy} - S_{xy}^2 - \left\lfloor \frac{K_{Q8}(S_{xx} + S_{yy})^2}{256} \right\rfloor \quad (4)$$

where $k=K_{Q8}/256$; the default $K_{Q8}=10$ gives $k=0.0390625$. Only the fractional bits of the k product are discarded, and the reference model applies the identical floor.

corner_threshold compares the signed response with the programmable threshold T, asserts `corner_out` when R>T, and passes on the coordinates and valid bit.

The design will be implemented with the Cadence flow: RTL simulation and lint; Genus synthesis with the course library and constraints; Conformal LEC between RTL and netlist; Tempus setup/hold analysis; Innovus floorplanning, power planning, placement, clock-tree synthesis and routing; and physical verification and GDSII stream-out.

Design Specifications

Tables 3–5 list the interface, the parameters and the arithmetic widths. The functional requirements are:

1. Accept one pixel per clock whenever `pixel_valid` is high; when it is low, all counters, buffers and pipeline registers hold their state.
2. Process pixels in raster order; after W× H accepted pixels, the next pixel starts a new frame automatically.
3. Assert `corner_valid` only for coordinates x∈[3,W-4] and y∈[3,H-4], where both the Sobel and the 5×5 windows are complete; images smaller than 7×7 produce no outputs.
4. Use a fixed latency of six register stages from a complete 3×3 window to the output, counted in accepted transactions.
5. Match the bit-accurate Python reference model exactly on every valid output coordinate and decision.
6. Synthesize without latches, use an asynchronous active-low reset, and meet the 10 ns clock target (0.2 ns uncertainty, 1 ns input/output delay) after routing.

Table 3: Top-Level Interface of `harris_top`

Port	Dir.	Width	Function
`clk`	in	1	System clock (10 ns target)
`rst_n`	in	1	Asynchronous active-low reset
`pixel_in`	in	8	Unsigned grayscale pixel
`pixel_valid`	in	1	Pixel strobe; low = stall
`corner_out`	out	1	1 when R > T
`corner_valid`	out	1	Output is a legal, complete window
`corner_x`	out	$\lceil \log_2 W \rceil$	Column of the decision
`corner_y`	out	$\lceil \log_2 H \rceil$	Row of the decision

Table 4: Design Parameters

Parameter	Default	Meaning
`IMAGE_WIDTH` (W)	16	Pixels per row; sets line-buffer depth
`IMAGE_HEIGHT` (H)	16	Rows per frame
`K_Q8`	10	Harris k = $K_{Q8}/256$ = 0.0390625
`THRESHOLD` (T)	10^6	Signed 54-bit response threshold

Table 5: Arithmetic Widths from Worst-Case Bounds (8-bit Input)

Quantity	Format	Range
I_x, I_y	signed 11-bit	±1,020
P_{xx}, P_{yy}	unsigned 20-bit	[0, 1,040,400]
P_{xy}	signed 21-bit	±1,040,400
5-row sums	23 / 24-bit	up to 5,202,000

Quantity	Format	Range
S_{xx}, S_{yy}	unsigned 25-bit	[0, 26,010,000]
S_{xy}	signed 26-bit	±26,010,000
det term	unsigned 50-bit	$<6.8\times10^{14}$
S_{xy}^2, tr^2	52-bit	$<2^{52}$
R	signed 54-bit	all K_{Q8} values

At the 100 MHz target the core processes 100 Mpixel/s, equivalent to about 325 fps for a 640×480 frame if built with W=640. Line storage is 2W×8 bits in `window3x3` plus 4W×(20+20+21) bits in `boxsum5x5`, that is 260W bits (4,160 bits at the default W=16).

Rationale and Scope

The survey directly informed the design. From [5], [8], [9], [10] we *adopt* the on-chip, row-buffered streaming organisation, since it is the only one that sustains one pixel per clock. From [5] we adopt computing only the three distinct entries of M and a small fixed-point k (our 10/256 equals their 5/128), and we turn their frame-boundary artefact into an explicit requirement for border masking. From [6], [7] we adopt the separation of window generation from computation and compile-time parameterisation of image size, k and threshold. From [2], [3], [12] we adopt per-node width analysis, but *adapt* it: version 1 keeps full precision so that the RTL is exactly equivalent to the reference model and the LEC and regression results are unambiguous, and threshold-guided truncation [3] is kept as a documented optimisation if timing or area require it. Finally, we *adapt* the Gaussian window of [11], [12] to a uniform 5×5 box sum, which removes all window multipliers at a small cost in isotropy.

In scope: the streaming datapath of Figure 1; parameterised image size, k and threshold; border and valid handling; a bit-accurate Python reference model with directed (flat, horizontal, vertical, diagonal, corner, checkerboard) and seeded random regression, including stall cycles; Verilator lint; Genus synthesis with area, timing and power reports; Conformal LEC; Tempus setup/hold analysis; Innovus floorplan, power plan, placement, CTS and routing; and DRC/connectivity checks and GDSII stream-out as supported by the course setup.

Out of scope for version 1: non-maximum suppression, adaptive per-frame thresholds [5], early non-corner gating for power [4], truncated arithmetic [3], SRAM macros for the line buffers, multiscale detection [10], and camera or display interfaces. These are recorded as extensions; the first two are the natural next step once post-route timing, area and power figures are available.

References

[1] R. Ghodhbani, T. Saidani, A. Alhomoud, A. Alshammari, and R. Ahmed, “Real Time FPGA Implementation of an Efficient High Speed Harris Corner Detection Algorithm Based on High-Level Synthesis,” *Engineering, Technology & Applied Science Research*, vol. 13, no. 6, pp. 12169–12174, 2023, doi: 10.48084/etasr.6306.

[2] P. Sikka, A. R. Asati, and C. Shekhar, “Real time FPGA implementation of a high speed and area optimized Harris corner detection algorithm,” *Microprocessors and Microsystems*, vol. 80, art. 103514, 2021, doi: 10.1016/j.micpro.2020.103514.

[3] B. A. Jasani, S.-K. Lam, P. K. Meher, and M. Wu, “Threshold-Guided Design and Optimization for Harris Corner Detector Architecture,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 28, no. 12, pp. 3516–3526, 2018, doi: 10.1109/TCSVT.2017.2757998.

[4] S.-K. Lam, R. K. Bijarniya, and M. Wu, “Lowering Dynamic Power in Stream-Based Harris Corner Detection Architecture,” in *Proc. Int. Conf. Field-Programmable Technology (ICFPT)*, 2017. [Online]. Available: https://www3.ntu.edu.sg/home/ASSKLam/Paper/2017D-Lam-ICFPT.pdf

[5] T. L. Chao and K. H. Wong, “An Efficient FPGA Implementation of the Harris Corner Feature Detector,” in *Proc. 14th IAPR Int. Conf. Machine Vision Applications (MVA)*, Tokyo, Japan, 2015, pp. 89–93. [Online]. Available: https://www.mva-org.jp/Proceedings/2015USB/papers/05-16.pdf

[6] A. Hernandez-Lopez, C. Torres-Huitzil, and J. J. Garcia-Hernandez, “FPGA-Based Flexible Hardware Architecture for Image Interest Point Detection,” *International Journal of Advanced Robotic Systems*, vol. 12, 2015, doi: 10.5772/61058.

[7] P. R. Possa, S. A. Mahmoudi, N. Harb, C. Valderrama, and P. Manneback, “A Multi-Resolution FPGA-Based Architecture for Real-Time Edge and Corner Detection,” *IEEE Transactions on Computers*, vol. 63, no. 10, pp. 2376–2388, 2014, doi: 10.1109/TC.2013.130.

[8] A. Amaricai, C.-E. Gavriliu, and O. Boncalo, “An FPGA Sliding Window-Based Architecture Harris Corner Detector,” in *Proc. 24th Int. Conf. Field Programmable Logic and Applications (FPL)*, Munich, Germany, 2014, pp. 1–4, doi: 10.1109/FPL.2014.6927402.

[9] M. F. Aydogdu, M. F. Demirci, and C. Kasnakoglu, “Pipelining Harris Corner Detection with a Tiny FPGA for a Mobile Robot,” in *Proc. IEEE Int. Conf. Robotics and Biomimetics (ROBIO)*, Shenzhen, China, 2013, pp. 2177–2184, doi: 10.1109/ROBIO.2013.6739792.

[10] P.-Y. Hsiao, C.-L. Lu, and L.-C. Fu, “Multilayered Image Processing for Multiscale Harris Corner Detection in Digital Realization,” *IEEE Transactions on Industrial Electronics*, vol. 57, no. 5, pp. 1799–1805, 2010, doi: 10.1109/TIE.2010.2040556.

[11] C. Harris and M. Stephens, “A Combined Corner and Edge Detector,” in *Proc. 4th Alvey Vision Conference*, Manchester, UK, 1988, pp. 147–151, doi: 10.5244/C.2.23.

[12] MathWorks, “Corner Detector: Find corners by using FAST or Harris algorithm,” Vision HDL Toolbox documentation. [Online]. Available: https://www.mathworks.com/help/visionhdl/ref/cornerdetector.html

[13] T. L. Chao, “FPGA_Harris_Corner: An efficient FPGA implementation of the Harris Corner feature detector,” GitHub repository. [Online]. Available: https://github.com/chaotaklon/FPGA_Harris_Corner

[14] A. Singh *et al.*, “ASIC_Proj: Streaming Harris Corner Detection ASIC,” project source repository (RTL, reference model, testbenches and Cadence scripts). [Online]. Available: https://github.com/ashmitsingh7/ASIC_Proj